Agent Evals Platform — Design Document

Oct 16, 2026 · @Vineethanand

Overview

The platform answers one question per experiment: does config B beat config A on this task set, by how much, at what cost, and is the difference real or noise. Everything else in this design serves that answer: isolated repeated trials, trustworthy graders, paired statistics, and traces detailed enough to explain a result.

Goals

- Evaluate any harness X with any model Y and settings S, where X and Y are plug-ins behind adapters.
- Produce paired comparisons with confidence intervals, per-task deltas, and an explicit "no detectable difference" verdict.
- Make every trial inspectable end to end and every result reproducible from a pinned config version.
- Run locally with `docker compose up` and keys in `.env`.

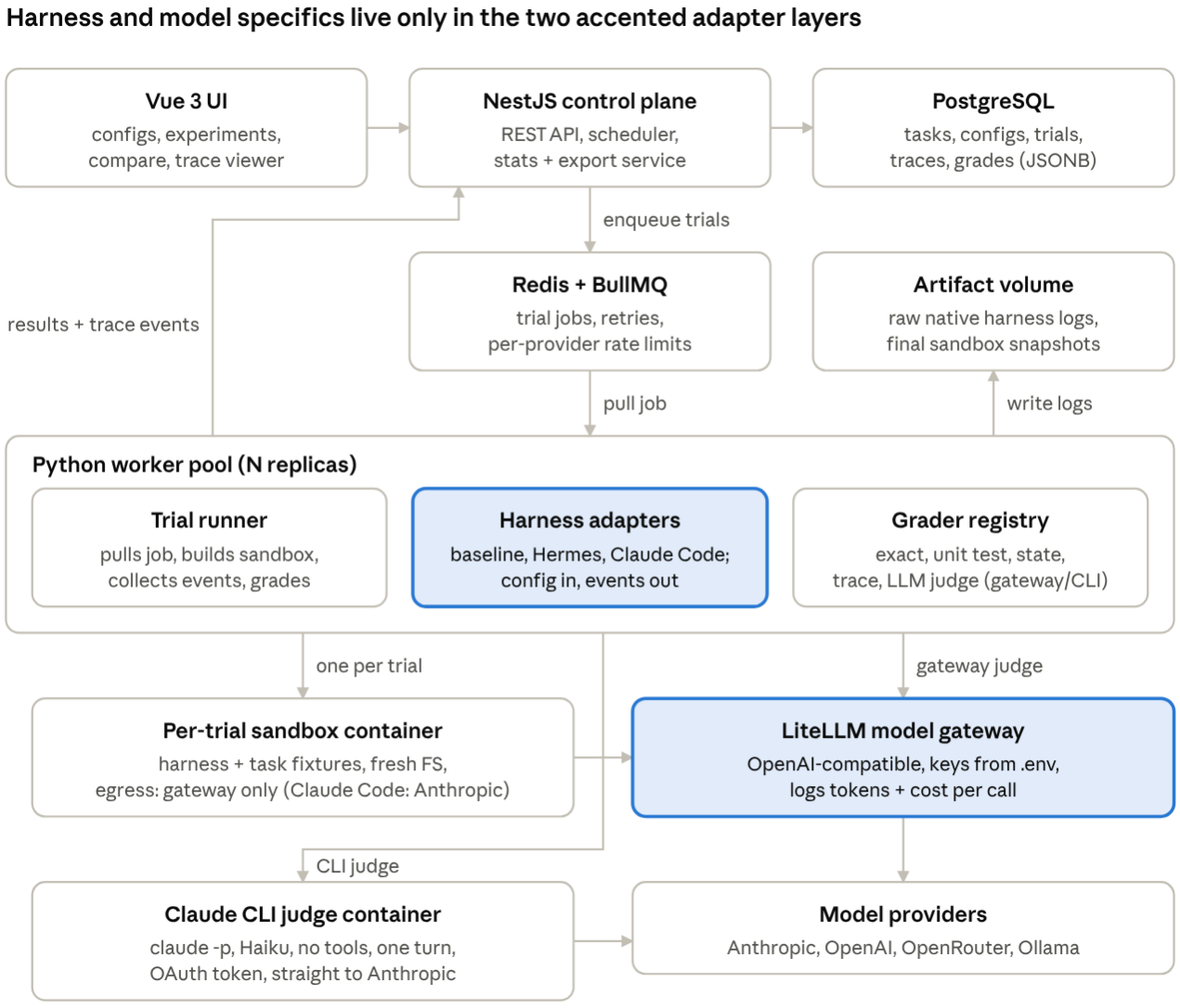
Key decisions at a glance

Decision	Choice	Why
Control plane	NestJS + PostgreSQL + BullMQ on Redis	Required stack; BullMQ gives retries, backoff, rate limiting and dead-lettering
Harness execution	Python workers, one Docker sandbox per trial	Hermes is Python; per-trial containers give real isolation
Model access	LiteLLM proxy as the single model gateway	One OpenAI-compatible endpoint for every provider; central token and cost accounting
Trace format	Normalised event stream (OpenTelemetry-style spans) stored as JSONB	One viewer for every harness; adapters translate native logs
Comparison stats	Bootstrap confidence interval plus a McNemar cross-check	Simple, paired, and asked for in the brief

Decision	Choice	Why
Report experiment	Hermes + memory on vs off, same model	Single clean axis; memory is where Hermes differs most from a baseline loop

Architecture

NestJS owns all state and decisions; Python workers only execute trials and report back. The split keeps the TypeScript side free of harness code and lets the worker pool scale by adding replicas.



system architecture · control plane, queue, workers, sandbox, gateway

A trial flows top to bottom: the API expands an experiment into trial jobs, a worker pulls one, launches a fresh sandbox, runs the harness through its adapter, grades the result, and posts the outcome and trace events back.

Component responsibilities

Component	Owns
NestJS API	Task/config registry, experiment expansion, job enqueueing, idempotent result ingestion, statistics, export
BullMQ (Redis)	Job state, retries with exponential backoff, per-provider concurrency groups, dead-letter queue
Python worker	Sandbox lifecycle, adapter invocation, event collection, grading, cost tally
Sandbox	One trial's filesystem, processes and memory store
LiteLLM gateway	Provider routing, key handling, token/cost logs keyed by `trial_id` header, 429 backoff
Vue UI	Config editor, experiment launcher, comparison view, trace viewer

Why one gateway for models. Every harness that can speak the OpenAI chat-completions protocol (Hermes, smolagents, the baseline loop, OpenAI Agents SDK) is pointed at `http://gateway:4000` with a per-trial virtual key. That gives one place to count tokens and dollars, enforce budgets, and inject the `trial_id` tag, independent of the harness.

Compose services: `api`, `ui`, `postgres`, `redis`, `gateway`, `worker` (scale with `--scale worker=4`). Harness images (`harness-hermes`, `harness-baseline`) are built by compose but run on demand by workers via the Docker socket.

Harness and model adapters

A new harness is one Python class plus one Dockerfile; a new model is one entry in the model registry. The platform core never branches on harness or model names.

Harness adapter contract (Python, runs inside the sandbox)

```
class HarnessAdapter(ABC):
    name: ClassVar[str]              # 'hermes', 'baseline'
    version: ClassVar[str]           # adapter version, recorded on every
trial
    capabilities: ClassVar[set[str]] #
{'skills','memory','tools','temperature','step_budget'}

    def prepare(self, ctx: TrialContext) -> None:
        """Materialise config into native form: write ~/.hermes config,
        install skills, seed or clear memory, register tools."""

    def run(self, ctx: TrialContext, emit: EventSink) -> RunResult:
        """Execute the task prompt to completion or budget exhaustion.
        Must call emit() for every normalised event as it happens."""

    def collect(self, ctx: TrialContext) -> Artifacts:
        """Return native logs, memory files after the run, workspace path."""
```

- `TrialContext` carries: task prompt and fixtures path, resolved config (model alias, temperature, seed, budgets, enabled tools/skills, memory mode), gateway URL and virtual key, `trial_id`, wall-clock deadline.
- `RunResult` carries: final answer text, stop reason (`completed`, `step_budget`, `token_budget`, `timeout`, `harness_error`, `gave_up`), step count.
- `EventSink` writes normalised events (next subsection). Adapters translate native logs into events; they never invent a format of their own.
- **Capability check.** A config that asks for `memory: on` on a harness without the `memory` capability is rejected at config validation time, not at run time. This keeps comparisons honest: a setting cannot silently do nothing.

Normalised trace event schema

Event type	Key fields
`llm_call`	model, request messages (hash + stored body), response, input/output/cached tokens, cost_usd, latency_ms
`tool_call`	tool name, arguments (raw JSON), schema-valid flag
`tool_result`	tool name, output (truncated at 16 KB, full in artifact), error flag, duration_ms

Event type	Key fields
`skill_invoked`	skill name, skill version hash
`memory_read` / `memory_write`	store (`MEMORY.md` , `USER.md` , session DB), key or diff
`agent_message`	role, content (thinking kept separately if exposed)
`harness_note`	free-form diagnostics, e.g. context compression or retries

Every event has `trial_id` , monotonic `seq` , `ts` , `parent_seq` (to nest tool results under calls) and `source` (`adapter` vs `gateway`). The gateway independently logs every LLM call; the trial's token and cost totals come from the gateway, not from the harness, so a harness that under-reports cannot look cheaper.

Hermes adapter

Hermes Agent is Nous Research's open-source agent with agent-curated memory and skills that it creates and refines during use (GitHub). It keeps memory in Markdown files (`MEMORY.md` , `USER.md`), session history in SQLite with FTS5 search, and writes skills as `SKILL.md` files under its skills directory (Better Stack guide). It can target any OpenAI-compatible endpoint, which is how it reaches the gateway.

- **Isolation:** each trial sets `HERMES_HOME` to a fresh directory inside the sandbox, so config, memory, sessions and skills cannot leak.
- **Invocation:** drive its Python agent class in-process (single query, non-interactive) rather than the TUI. Assumption to verify on day 1: the exact entry point and whether it exposes a step cap; if not, the adapter enforces the step budget by counting `llm_call` events and cancelling.
- **Skills:** the config lists skill bundles from `tasks/skills/` ; `prepare()` copies them into the skills directory. Self-authored skills created during a trial are captured in `collect()` and discarded unless memory mode persists them.
- **Memory modes:** `off` (memory tools disabled, files empty), `fresh` (enabled, empty at start), `seeded:<fixture>` (enabled, preloaded files), `carry:<group>` (state persists across trials in an ordered group; see Execution).
- **Trace source:** Hermes's own trajectory output plus tool hooks, translated to events. Memory reads and writes are detected from its memory tool calls and confirmed by diffing the memory files before and after.

Second harness: baseline tool-calling loop

A 150-line plain loop: system prompt, tools as OpenAI function schemas, call model, execute tools, repeat until a final answer or budget. It has `tools`, `temperature` and `step_budget` capabilities, and a simple `memory` capability (a JSON notes file exposed as `notes_read` / `notes_write` tools), so memory experiments can run on both harnesses. Writing it ourselves gives a fully transparent control: every behaviour difference against Hermes is attributable to Hermes. A third adapter for smolagents is a stretch goal to prove the interface again.

Model adapter

Models are rows in `models.yaml`, loaded into the LiteLLM gateway at startup and mirrored in the DB:

```
- alias: claude-sonnet
  litellm_model: anthropic/claude-sonnet-4-5
  price: { input_per_mtok: 3.00, output_per_mtok: 15.00 }
  supports: { tools: true, temperature: true, seed: false }
- alias: qwen3-open
  litellm_model: openrouter/qwen/qwen3-235b-a22b
  price: { input_per_mtok: 0.20, output_per_mtok: 0.60 }
  supports: { tools: true, temperature: true, seed: true }
```

Adding a model is adding a row. If a provider is not in LiteLLM, the escape hatch is a custom LiteLLM provider class, still outside the platform core. Prices are snapshotted into each config version so later price changes do not rewrite historical cost. Model IDs and prices above are placeholders to confirm against provider pages at build time.

Adding things: the whole diff

To add	You write	Core changes
Harness	`workers/adapters/<name>.py`, `harnesses/<name>/Dockerfile`, register in `adapters/__init__.py`	None
Model	One `models.yaml` entry (+ key in `.env`)	None
Task	One folder under `tasks/<id>/` (see Task set)	None
Grader type	One class implementing `Grader.grade()`	None

Data model

Everything a result depends on is immutable and content-addressed: task versions, config versions and grader versions are hashed, and a trial points at exact versions, never at a mutable "latest".

Relationships: a **Task** has many **TaskVersions**; a **Config** has many **ConfigVersions**; an **Experiment** pins a task-set snapshot and 2+ config versions and expands into **Trials** (task version × config version × trial index); a Trial has many **TraceEvents**, one or more **Grades**, and at most one **FailureLabel**. **JudgeCalibration** rows hold human labels used to validate judges.

Entity	Key fields
`task`	id (slug), title, tags (`memory-sensitive` , `skill-sensitive` , `tool-use` , `reasoning`)
`task_version`	task_id, content_hash, prompt, fixtures_uri, grader_spec (JSON), timeout_s, created_at
`task_set`	id, name, task_version_ids[], hash
`config`	id, name, description
`config_version`	config_id, version (int), hash, harness, harness_image_digest, adapter_version, model_alias, model_snapshot (resolved provider id + prices), settings JSON, created_at
`experiment`	id, name, hypothesis, task_set_id, config_version_ids[], n_trials, seed, status, primary_metric, created_at
`trial`	id, experiment_id, task_version_id, config_version_id, trial_index, seed, status, attempt, stop_reason, final_answer, score, passed, tokens_in/out/cached, cost_usd, latency_ms, steps, error, started/finished_at
`trace_event`	trial_id, seq, parent_seq, type, ts, payload JSONB
`grade`	trial_id, grader_id, grader_version, score (0..1), passed, reasoning, raw JSONB, judge_tokens, judge_cost_usd
`failure_label`	trial_id, category, source (`rule` / `llm` / `human`), evidence_seq[]

Entity	Key fields
`judge_calibration`	grader_id, grader_version, example_id, human_label, judge_label, judge_reasoning

Config settings JSON (validated with a shared JSON Schema used by both NestJS and the worker):

```
{
  "temperature": 0.7,
  "seed": 1234,
  "max_steps": 25,
  "max_tokens_total": 200000,
  "timeout_s": 600,
  "tools": ["bash", "read_file", "write_file", "web_fetch_stub"],
  "skills": ["csv-report@1", "git-hygiene@1"],
  "memory": { "mode": "carry", "group_by": "task.memory_group" },
  "system_prompt_override": null
}
```

Reproducibility rule: a past result is reproduced by re-running its experiment with the same task-set hash, config-version hashes, harness image digest and seeds. Exact token-level replay is impossible with hosted models; reproduction means the same distribution, checked by the comparison engine reporting no significant difference between the original and the re-run.

Execution, isolation and failure handling

Each trial runs in its own throwaway container, and a trial's result is written exactly once or marked as an infrastructure failure; it is never half-written and never silently dropped.

Expansion and scheduling

1. `POST /experiments` validates configs against harness capabilities, freezes the task set, and computes `tasks × configs × N` trial rows with status `queued`.
2. Trials are enqueued **interleaved**: trial index is the outer loop and configs alternate inside it. Provider slowdowns or outages then hit both configs roughly equally instead of biasing one.
3. Each job carries only `trial_id`; the worker fetches the frozen spec from the API, so a retried job always runs the same spec.
4. BullMQ group concurrency is keyed by provider (e.g. `anthropic: 4`, `openrouter: 8`) to stay under rate limits.

Trial lifecycle

1. Worker claims the job and calls `POST /internal/trials/:id/start` (sets `running`, records `attempt`).
2. Creates the sandbox: `docker run` from the harness image digest, with a fresh tmpfs workspace, the task fixtures copied in, CPU 1 / RAM 2 GB, network `evalnet` that only routes to the gateway.
3. Mints a per-trial LiteLLM virtual key with `max_budget` set from the config's token budget priced in dollars.
4. Runs `prepare → run → collect` under a wall-clock deadline; events stream to the API in batches of 50.
5. Grades inside a separate grader container that mounts the final workspace read-only (graders never share a process with the agent).
6. `POST /internal/trials/:id/complete` with an idempotency key `(trial_id, attempt)`; the API upserts and ignores duplicates.
7. Tears down the container and volume, always, in a `finally` block.

Isolation rules

State	Default	Exception
Filesystem	Fresh per trial	None
Harness home (`HERMES_HOME`, notes file)	Fresh per trial	`memory.mode = carry`
Memory and skills learned during a run	Discarded	`carry` keeps them for the next trial in the same carry chain
Network	Gateway only	Tasks that need a fake web get a stub HTTP server inside the sandbox
Model cache	Gateway response cache **off** for agent calls	On for judge calls with identical inputs (cost saving, no effect on agent behaviour)

Carry-over memory without breaking statistics. Memory tasks come in ordered *chains*: e.g. chain M1 has a "teach" task (user states a preference or the agent discovers a fact) followed by 1–2 "use" tasks. A carry chain replays the whole sequence inside one persistent home directory, and each of the N trials gets its own independent chain. Only the "use" tasks are scored for the memory comparison. So trials stay independent across trial indices while memory persists within a chain, exactly as the setting says.

Failure handling

Failure	Detection	Handling	Counts as
Provider 429 / 5xx	Gateway	LiteLLM retries with exponential backoff (3 tries, jitter); queue group concurrency throttles	Not a failure if it recovers
Agent timeout	Wall-clock deadline	Kill container, keep events so far, stop_reason `timeout`	Agent failure (scored 0)
Step or token budget hit	Adapter counter / gateway `max_budget`	Stop gracefully, grade whatever state exists	Agent failure if grader fails it
Harness crash (adapter exception)	Non-zero exit	Retry once; then status `harness_error`	Infra failure, excluded from stats and reported
Worker dies mid-trial	BullMQ stalled-job check (lock expiry 60 s)	Job re-queued with attempt+1; old partial events discarded by attempt number	Retried
Persistent provider outage	Retries exhausted	Job to dead-letter queue, status `infra_error`	Excluded; experiment marked `incomplete`
Judge call fails	Grader exception	Retry 3×, then grade status `ungraded`	Excluded from that grader's stats, flagged

Rule for exclusions. Agent-caused failures (timeouts, budgets, bad answers) always count as 0; infrastructure failures are excluded. Paired statistics then drop the matching cells from both configs. If more than 5% of cells for any config are infra failures, the comparison page shows a red banner and the verdict is withheld until the experiment is resumed (`POST /experiments/:id/resume` re-queues only failed cells).

Task set and graders

The sample set has 24 tasks: 21 scored and 3 unscored teach tasks. 20 scored tasks are graded deterministically and 1 by an LLM judge; 5 are built so that a skill or memory should change the outcome.

Task format

One folder per task; adding a task means adding a folder and running `pnpm task:validate <id>`.

```
tasks/csv-aggregate/
  task.yaml          # prompt, tags, timeout, grader spec, memory_group
(optional)
  fixtures/          # copied into the sandbox workspace
  grader/check.py    # for state_check / unit_test graders
  reference/         # a known-good solution (must pass the grader)
  negatives/         # 1-2 known-bad solutions (must fail the grader)
```

```
id: csv-aggregate
prompt: |
  sales.csv is in your workspace. Write report.json with total revenue per
region,
  rounded to 2 decimals, regions sorted alphabetically.
tags: [tool-use, data]
timeout_s: 300
graders:
  - type: state_check
    primary: true
    script: grader/check.py
  - type: trace_check        # secondary, diagnostic only
    assert: [{ tool_used: read_file }]
```

Grader types

Grader	Input	Output	Used for
`exact_match`	Final answer	One check. Normalisers (`trim`, `lower`, `collapse_ws`, strip punctuation) then a mode: `equals`, `contains`, or `number` with a tolerance. `expected` may list several accepted answers	Lookups, numeric answers, the teach tasks
`unit_test`	A copy of the final workspace plus hidden tests (`grader/tests`)	Runs `python -m unittest`; score = share of tests passed, pass = all pass	Coding tasks; the agent never sees the tests

Grader	Input	Output	Used for
`state_check`	Final workspace and final answer	The task's own `grader/check.py` returns named checks (`must` by default); pass = every must-check passes	File, config, data and git-state tasks (most of the set)
`trace_check`	Normalised trace events	Assertions: `tool_used`, `tool_not_used`, `event` (e.g. `memory_read`), `command_contains`	Memory and skill tasks: did the agent use what it learned?
`llm_judge`	Task prompt, rubric, final answer, files the agent produced	Per-criterion reason then yes/no; pass = every `must` is yes. Backend: the LiteLLM gateway (`JUDGE_MODEL`) or the `claude` CLI (`JUDGE_BACKEND=claude-cli` or `backend: claude-cli` on the grader; model `JUDGE_CLI_MODEL`, default Haiku)	Open-ended writing

Every grader returns the same shape: `score ∈ [0,1]`, `passed: bool`, `reasoning`, `checks[]`. The primary metric for comparisons is **pass rate** on the primary grader; partial scores are reported but not tested, because binary outcomes make McNemar and pass-rate CIs straightforward.

Validating deterministic graders

Graders are code, so they get tests. CI runs every grader against its `reference/` (must pass) and `negatives/` (must fail). A task cannot be merged if either check fails. This catches the most common eval bug: a grader that passes everything or nothing.

LLM judge design

- **Judge model** from a different provider family than the agent models in the experiment where possible, to limit self-preference; temperature 0; pinned version stored in the grader version.
- **Rubrics decomposed** into 4–6 binary criteria, each marked `must` or `nice`. Binary criteria are easier to agree on than a 1–10 scale.
- **Judge sees** the task, rubric, final answer and workspace diff. It does not see the agent's reasoning or self-assessment, so it cannot be argued into a pass.

- **Output** is JSON: per-criterion `reasoning` written before `verdict`. Malformed JSON is retried, then marked `ungraded`.
- **Two backends.** By default the judge prompt goes through the LiteLLM gateway like any model call. With `claude-cli` the worker runs `claude -p <prompt> --output-format json --max-turns 1` with every tool disabled, authenticated by `CLAUDE_CODE_OAUTH_TOKEN` (from `claude setup-token`). In Docker mode this runs in a throwaway container from the `claude-code` harness image on the egress network, with no mounts, so the judge sees only the prompt. Token usage comes from the CLI's JSON result and is added to the trial's judge cost. One experiment uses one judge for every trial.

Validating the judge

1. Run a pilot (both experiment configs, N=3) on the 5 judged tasks; sample 60 outputs stratified across tasks and configs, roughly half passes.
2. Label them by hand against the same rubric, blind to config and judge verdict. Re-label 15 a week later to measure my own consistency.
3. Split 30 dev / 30 held-out. Tune rubric wording on dev only; report on held-out.
4. Report accuracy, Cohen's κ with a bootstrap 95% CI, and false-pass rate (judge passes what a human failed).
5. Bias probes: re-judge 20 examples with the answer padded by irrelevant but fluent text (length bias) and with config labels swapped in metadata (should change nothing). Re-run each example 3× to measure verdict flip rate.
6. **Acceptance bar:** κ ≥ 0.7 and false-pass rate ≤ 10% on held-out. A judge that misses the bar is still run, but its tasks are excluded from the headline verdict and shown separately with a warning.

The calibration set lives in `judge_calibration` and is re-scored automatically whenever a rubric or judge model changes, so judge drift is visible.

Statistical method and choosing N

We compare the pass rates of config A and config B, give a 95% confidence interval for the difference, and check it with one paired test. Each task runs 10 times per config.

What we measure

- **Pass rate** = trials passed ÷ trials run, for each config.
- **Difference** = pass rate of B − pass rate of A, in percentage points. Example: A 60%, B 72% → +12 points.

- Cost, tokens and time per trial are compared the same way.

Why the comparison is paired

A and B run the **same tasks with the same seeds**. Some tasks are hard for everyone, some easy. Comparing A and B on the same task cancels that out, so the only thing left is the effect of the config.

Confidence interval: bootstrap

The bootstrap shows how much the difference would move if we had picked a slightly different set of tasks.

1. Pick 21 tasks at random from our 21 scored tasks, allowing repeats. Keep each task's A and B results together.
2. Recompute B − A on that pick.
3. Repeat 10,000 times.
4. The middle 95% of those 10,000 values is the confidence interval.

We resample **tasks**, not individual trials. The 10 trials of one task are closely related, so counting them as 10 independent results would make us overconfident.

Paired test: McNemar

1. For each task, mark a config as "pass" if it passed most of its trials.
2. Ignore tasks where A and B agree (both pass or both fail). They say nothing about which is better.
3. Count the tasks where only B passed and only A passed.
4. If A and B were equal, each of those would be a coin flip. McNemar gives the chance of a split at least that lopsided. Below 5% ($p < 0.05$) counts as a real difference.

The bootstrap is the main result; McNemar is a cross-check. If they disagree, we report both.

Reading the result

Confidence interval	What we say
Entirely above 0	B is better
Entirely below 0	A is better
Crosses 0	No clear difference: could be noise

As a sanity check, we also run one config against itself once. It should come out as "no clear difference"; if it doesn't, something in the platform is broken.

Choosing N

Agents are random, so one trial per task tells us little. More trials help, but only up to a point. With 21 scored tasks, the smallest difference we can reliably detect shrinks like this:

Trials per task (N)	Smallest difference we can detect	Relative cost
3	about 27 points	0.3×
5	about 21 points	0.5×
10	**about 16 points**	**1×**
20	about 13 points	2×

Going from 10 to 20 doubles the cost for a small gain; adding more tasks would help more. So we:

1. Run a small pilot with N=3 to see how much results vary.
2. Use **N=10** for the main run.
3. Use N=20 on the few memory tasks, because the main hypothesis rests on only 3 tasks.

Numbers above assume tasks differ moderately in how much the change helps them; the pilot replaces that assumption with measured values.

Run inspection and failure analysis

Any trial opens to a single page that answers "why did it pass or fail" without leaving the browser: the verdict and grader reasoning at the top, the full timeline below, and a failure category that links to the exact events behind it.

What the trial view shows

The page is titled with the task id and trial index. A summary table gives the trial ID, the config (harness, model, memory mode and version), the result (status, pass or fail, stop reason and failure category), the cost with input and output tokens, steps and latency, the seed and attempt number, any error message, and the agent's final answer. A link opens the trial with the same index from the other config, so the pair can be compared. Below that are the grades (each grader's pass or fail, score and named checks with their details),

a **Show log files** button that loads the raw files the worker saved for the trial (harness log, container log, `worker_error.txt`), the task prompt, and the timeline: every event in order (model calls with token counts, tool calls nested under the call that made them, tool results, memory reads and writes, skill use and harness notes), each one expandable to its full payload.

Failure taxonomy

Every failed trial gets exactly one primary category. Rules run first because they are cheap and auditable; an LLM classifier runs only on what rules leave as `other`, and its labels are marked as such.

Category	Rule (deterministic)	Evidence linked
Infra error	status `infra_error` or `harness_error`	Error event
Step or budget exhaustion	stop_reason `step_budget` / `token_budget` / `timeout`	Last 3 events
Loop	Same tool with identical arguments ≥ 3 times in a row	Repeated calls
Hallucinated tool or args	Unknown tool name, or arguments fail the tool's JSON schema	The invalid call
Wrong tool	Task declares `required_tools` and none were called, or a forbidden tool was called	Tool calls
Gave up early	Completed with < 25% of the step budget and a final answer containing refusal or uncertainty phrases, and grader failed	Final message
Ignored memory or skill	Memory/skill task where the needed memory read or skill invocation never happened	Missing event (shown as absent)
Wrong answer	Ran to completion with valid tool use, grader failed	Failed checks
Grader disagreement	Two graders on the same trial disagree (e.g. regex gate passes, judge fails), or judge flipped across re-runs	Both grades

Category	Rule (deterministic)	Evidence linked
Other	Nothing above matched; sent to LLM classifier	Classifier reasoning

The comparison page shows a **failure-category diff**: counts per category for A and B side by side, plus which category moved most. A regression then reads as "B lost 9 points, 7 of them from loops on the coding tasks", not just "B lost 9 points".

The classifier is validated the same way as the judge, on 40 hand-labelled failures, and its category counts are shown with its measured accuracy so nobody over-reads them.

API and UI

The UI has six screens. Everything it shows is computed by the API, so exports and screens never disagree.

REST API (NestJS modules)

Module	Endpoints
Tasks	`GET /tasks`, `GET /tasks/:id/versions`, `POST /tasks/sync` (re-scan `tasks/`, create versions for changed hashes), `POST /task-sets`
Harnesses and models	`GET /harnesses` (with capabilities), `GET /models`
Configs	`GET /configs`, `POST /configs`, `POST /configs/:id/versions`, `GET /configs/:id/versions/:v`, `GET /configs/:id/diff?from=&to=`
Experiments	`POST /experiments`, `GET /experiments/:id` (progress), `POST /experiments/:id/resume`, `POST /experiments/:id/cancel`, `POST /experiments/:id/rerun` (reproduction)
Results	`GET /experiments/:id/comparison?a=&b=` (headline, per-task, cost, failure diff), `GET /experiments/:id/export?format=json`
Trials	`GET /trials/:id`, `GET /trials/:id/events?after=seq`, `GET /trials/:id/artifacts`

Module	Endpoints
Judges	`GET /graders/:id/calibration`, `POST /graders/:id/calibrate`
Internal (workers only, shared-secret header)	`GET /internal/trials/:id/spec`, `POST /internal/trials/:id/start`, `POST /internal/trials/:id/events`, `POST /internal/trials/:id/complete`

Request and response DTOs are generated from the shared JSON Schemas (`packages/schemas`) into both TypeScript (class-validator DTOs) and Python (pydantic models), so the worker and API cannot drift.

UI screens (Vue 3 + TypeScript + Vite)

1. **Home:** what the platform does, the four steps of a run (configs, experiment, grading, comparison), links to each screen and a key to the trial dot colours.
2. **Configs:** create a config by picking a harness (baseline, Hermes, Claude Code; each shows its capabilities) and a model. The model list includes every model installed in a local Ollama, labelled free and offline, with prices for the rest. Settings are JSON (`memory.mode` off, fresh or carry). **Edit** saves a new version; earlier versions and the experiments that used them stay untouched. Each config lists its versions with harness, model, settings and version id.
3. **Experiments:** every experiment with its hypothesis, status, progress (done of total, errors, cancelled), N and creation time, refreshed every 3 seconds. **Stop** cancels queued and running trials; **Delete** removes the experiment with its trials, events, grades and log folders.
4. **New experiment:** name (a default "A vs B" name if left blank), hypothesis, two config versions (first is A, second is B), trials per task, per-task N overrides, seed and task picker. Before launch it shows the trial count and an estimated cost.
5. **Comparison (experiment page): Stop and clear queue** and **Resume failed / stopped trials** with live counts; a task filter (all scored, memory only, all except memory, file-edit only, reply-only); CSV and JSON export; the verdict banner with Δ and 95% CI; quality and cost for A and B; the McNemar 2×2 table; per-task results with a CI bar and Fisher p; failure categories for A and B; an errors panel; and a grid of all trials as coloured dots that open the trial.
6. **Trial view:** described above under Run inspection.

Exports: trial-level CSV (one row per trial, flat columns), task-level CSV (per task × config pass rates and CIs), and a full JSON bundle (experiment, config versions, task versions, trials, grades, comparison) that is sufficient to recompute every number offline.

Cost model

The main experiment should cost roughly $40 on a mid-priced hosted model and under $10 on an open-weight model. Real spend is read from the model gateway's logs after the run.

Formula

Cost = tasks × trials × configs × average tokens per trial × price per token, plus a small extra for LLM-judge calls.

Worked estimate

Item	Value
Tasks	24
Trials per task	10
Configs	2
Total trials	24 × 10 × 2 = 480
Tokens per trial (assumed)	about 40,000 in, 3,000 out
Price (example mid-priced model)	$1 per million input, $5 per million output
Cost per trial	about $0.04 in + $0.015 out ≈ $0.05
Main run	**480 × $0.05 ≈ $24**
Pilot + self-check run + judge calls	about $15
Total estimate	**about $40**
Actual spend	to fill in from gateway logs

Token counts and prices are assumptions; the pilot run measures the real token averages.

How to reduce cost

- **Caching:** reuse the provider's prompt cache for the repeated system prompt and tools, and never pay twice for the same judge call.
- **Early stopping:** check results halfway; if the difference is already very clear, stop the run.
- **Sampling:** run fewer trials on tasks that always pass or always fail, since repeats there tell us nothing new.

- **Smoke test first:** try a new config on a cheap model with 1 trial to catch broken setups before a full run.
- **Budget cap:** the gateway stops a trial or experiment that goes over its dollar limit.

Experiment plan: Hermes memory on vs off

The report experiment changes one axis only: Hermes with the same model, settings and seeds, memory `carry` vs memory `off`. Hypotheses, metrics and stopping rules are written into the experiment record before the main run starts.

The platform is not tied to this axis: any config is a harness X plus a model Y plus settings, so the same flow compares two models in one harness (e.g. Hermes + Claude vs Hermes + an open-weight model) or two harnesses with one model. Only the two config cards change; tasks, graders and statistics stay the same. A two-model comparison is a valid alternative experiment and the fallback if memory can't be isolated in time.

Hypothesis

Written down before the main run:

1. **Memory helps where it is needed.** On the 3 memory tasks, memory on passes clearly more often.
2. **Memory does no harm elsewhere.** On the other tasks, pass rates stay about the same.
3. **Memory costs a bit more.** Reading and writing memory adds roughly 10–25% more tokens.

Setup

Item	Value
Config A	Hermes, memory off
Config B	Hermes, memory on (kept only within each teach → use pair of tasks)
Same in both	Model, temperature, step limit, time limit, skills, seeds
Tasks	21 scored tasks (+3 teach)
Trials	Pilot: 3 per task. Main: 10 per task, 20 on the 3 memory tasks
Total main-run trials	about 540

Tested experiment

A completed run, with its hypothesis, setup, results, per-task table, trajectories and conclusion, is written up in the repository at `docs/experiment-report.md` (baseline + local Qwen 2.5 7B vs Claude Code + Claude Haiku, memory off). This design document is kept in the same `docs/` folder.

Known limits

Results hold for these tasks, these graders and this run. Don't treat them as general truths about a harness or model.

Limit	What it means	What we do about it
Small task set	21 scored tasks can't detect small differences (under about 16 points)	Say so next to every result
Hand-built tasks	Memory and skill tasks were designed to show an effect, so they exaggerate real-world impact	Report them separately from the other tasks
Judge bias	An AI judge can favour longer answers or its own model's style	Use a judge from a different provider; check it against 60 human grades
Contamination	A model may have seen similar tasks in training	Write all tasks fresh with made-up data
Hermes quirks	Hermes can save memory or skills on its own, even when not asked	Fresh folder for every trial; record what it saved
Model changes	A provider can update a model behind the same name	Pin exact model versions; re-run the self-check
Randomness	Re-running never gives identical outputs	"Reproduced" means the same results within the confidence interval

Assumptions

- Hermes can be driven non-interactively from Python and pointed at an OpenAI-compatible base URL; to verify on day 1.

- Hermes's memory and skills can be confined to a per-trial home directory via `HERMES_HOME` or its config.
- LiteLLM's proxy covers the chosen providers and reports token usage, including cached tokens.
- Docker is available on the reviewer's machine and workers may use the Docker socket.
- Pass/fail is the right primary metric; partial credit is shown but not tested.
- Equal weight per task is the right aggregate; weights are a config option but default to equal.
- Single user, local only: no auth on the public API; internal API protected by a shared secret.
- Hermes with Ollama and a free model tier is tested, and it is expected to work with other models.